

Συστήματα Ανάκτησης Πληροφοριών (INF248)

Αναφορά Προγραμματιστικής Εργασίας – Φάση 3: Υβριδικό σύστημα ανάκτησης: συνδυασμός κλασικής και σημασιολογικής ανάκτησης

Ρήγας Σάσσαλος

Αρ. Μητρώου: 3220178

Στην τρίτη και τελευταία φάση της εργασίας, υλοποίησα ένα Υβριδικό Σύστημα Ανάκτησης το οποίο συνδυάζει την ταχύτητά και ακρίβεια της Φάσης 1 με την εννοιολογική κατανόηση της Φάσης 2. Η βασική ιδέα είναι η χρήση της *ElasticSearch* ως ενός αρχικού, γρήγορου φίλτρου για την ανάκτηση ενός υποσυνόλου σχετικών εγγράφων, τα οποία επαναταξινομούνται χρησιμοποιώντας *Transformers* και *FAISS*.

Η υλοποίηση της υβριδικής προσέγγισης ακολουθεί μια *αρχιτεκτονική δύο σταδίων*:

Στάδιο 1: Αρχική ανάκτηση με *ElasticSearch*

Για κάθε ερώτημα από το *queries.csv*, εκτελούμε μια αναζήτηση στο ευρετήριο *ir2025_documents (BM25)* της *ElasticSearch*. Ζητάμε από την μηχανή να μας επιστρέψει μόνο τα $N = 200$ πιο σχετικά κείμενα. Με αυτό τον τρόπο μειώνουμε τον όγκο των δεδομένων που πρέπει να επεξεργαστούμε σημασιολογικά, κερδίζοντας ταχύτητα και πόρους.

Στάδιο 2: Επαναταξινόμηση με *Transformers* and *FAISS*

Στη συνέχεια επεξεργαζόμαστε μόνο τα $N = 200$ κείμενα που επέστρεψε η *ElasticSearch*:

Χρησιμοποιούμε το μοντέλο *all-MiniLM-L6-v2* για να μετατρέψουμε τα 200 κείμενα σε διανύσματα. Αφού κανονικοποιήσουμε τα διανύσματα (*L2 Normalization*), δημιουργούμε δυναμικά (για κάθε ερώτημα ξεχωριστά) ένα προσωρινό ευρετήριο *FAISS* τύπου *IndexFlatIP*. Τέλος, εισάγουμε τα διανύσματα των κειμένων στο *FAISS* και εκτελούμε αναζήτηση *Cosine Similarity* για να βρούμε τα κορυφαία $k = 50$ έγγραφα.

Χρησιμοποίησα την εντολή `os.environ['KMP_DUPLICATE_LIB_OK']='True'` για την αποφυγή συγκρούσεων της βιβλιοθήκης *OpenMP* στο περιβάλλον *MacOS*.

```
with open(output_file, "w") as f:
    for index, row in df_queries.iterrows():
        q_id = row['ID']
        q_text = row['Text']

        # Retrieval ElasticSearch (Top N=200)
        res = es.search(
            index=index_name,
            query={'match': {'text': q_text}},
            size=N
        )
        hits = res['hits']['hits']

        # If elasticsearch does not find anything continue
        if not hits:
            continue

        # collect IDs and texts
        doc_ids = [hit['_id'] for hit in hits]
        doc_texts = [hit['_source']['text'] for hit in hits]

        # Create embeddings with Transformers
        q_emb = model.encode(q_text, convert_to_numpy=True).astype('float32')
        doc_embs = model.encode(doc_texts, convert_to_numpy=True).astype('float32')

        # Cosine Similarity
        faiss.normalize_L2(q_emb)
        faiss.normalize_L2(doc_embs)

        # Dynamic FAISS index
        dim = doc_embs.shape[1]
        faiss_index = faiss.IndexFlatIP(dim)
        faiss_index.add(doc_embs)

        # Search and Re-Rank
        # top k=50
        search_k = min(k, len(doc_ids))
        distances, indices = faiss_index.search(q_emb, search_k)

        # Write in file
        for rank, faiss_idx in enumerate(indices[0]):
            real_doc_id = doc_ids[faiss_idx]
            score = distances[0][rank]

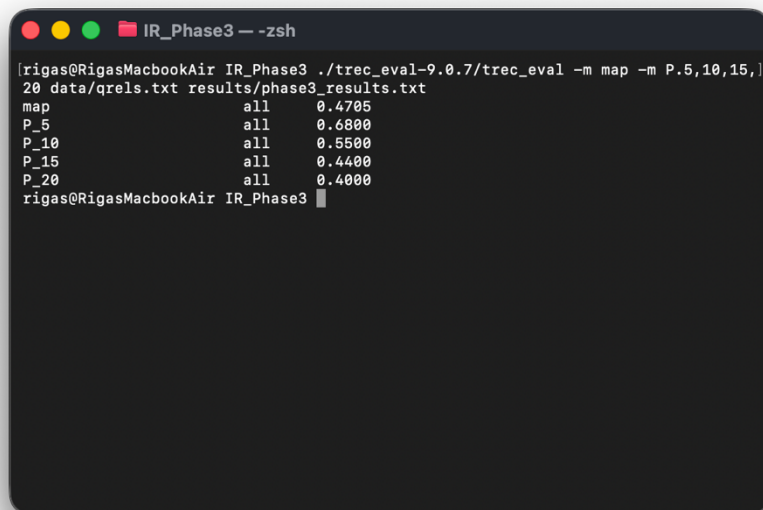
            f.write(f"{q_id} @ {real_doc_id} {rank+1} {score:.4f} {run_name}\n")

print(f"Hybrid Search Complete! Results saved to {output_file}")
```

✓ 16.8s Python

Hybrid Search Complete! Results saved to results/phase3_results.txt

Τα τελικά αποτελέσματα αποθηκεύτηκαν στο αρχείο *phase3_results.txt* στην απαιτούμενη μορφή *TREC* και αξιολογήθηκαν με το εργαλείο *trec_eval* σε σύγκριση με τα *qrels.txt*.



```
rigas@RigasMacbookAir IR_Phase3 --zsh
rigas@RigasMacbookAir IR_Phase3 ./trec_eval-9.0.7/trec_eval -m map -m P.5,10,15,|
20 data/qrels.txt results/phase3_results.txt
map          all      0.4705
P_5          all      0.6800
P_10         all      0.5500
P_15         all      0.4400
P_20         all      0.4000
rigas@RigasMacbookAir IR_Phase3
```

	Φάση 1 (BM25)	Φάση 2 (Semantic)	Φάση 3 (Hybrid)
MAP	0.7446	0.3741	0.4705
P@5	0.8600	0.6600	0.6800
P@10	0.7300	0.5000	0.5500
P@15	0.6867	0.3933	0.4400
P@20	0.6200	0.3500	0.4000

Παρατηρούμε ότι η υβριδική προσέγγιση προσφέρει μια ισορροπία μεταξύ ταχύτητας και ποιότητας αποτελεσμάτων. Επιλύει το βασικό πρόβλημα της Φάσης 2 όπου η αναζήτηση δυσκολευόταν να εντοπίσει συγκεκριμένα ακρωνύμια αφού η *ElasticSearch* φέρνει εγγυημένα τα κείμενα που περιέχουν αυτά τα ακρωνύμια στο $N = 200$. Στη συνέχεια απλά ο *Transformer* τα αναδιατάσσει βάζοντας ψηλά αυτά που είναι πιο συναφή.

Η υβριδική προσέγγιση είναι εξαιρετικά πιο γρήγορη σε σχέση με την Φάση 2 αφού αντί να συγκρίνουμε το ερώτημα με 18.316 διανύσματα στο *FAISS*, το συγκρίνουμε μόνο με 200 εξοικονομώντας μνήμη και χρόνο.

Μοναδικός περιορισμός της υβριδικής προσέγγισης είναι ότι εξαρτάται απόλυτα από το πρώτο βήμα. Αν ένα πολύ σχετικό έγγραφο δεν περιέχει της ακριβείς λέξεις-κλειδιά και η *ElasticSearch* δεν το συμπεριλάβει στα $N = 200$, το *FAISS* δεν θα έχει ποτέ την ευκαιρία να το βαθμολογήσει, που επηρεάζει το recall.